Progress Report 3 – Functional Design

Urban Scout – Unified Mobile Tourist Assistant

CPSC 471 – Winter 2026 – Group 61

Team members:

1) Sean Kim
2) Muhtasim Isham
3) Talha Hussain Mahr

Section 1: UML Class Diagram

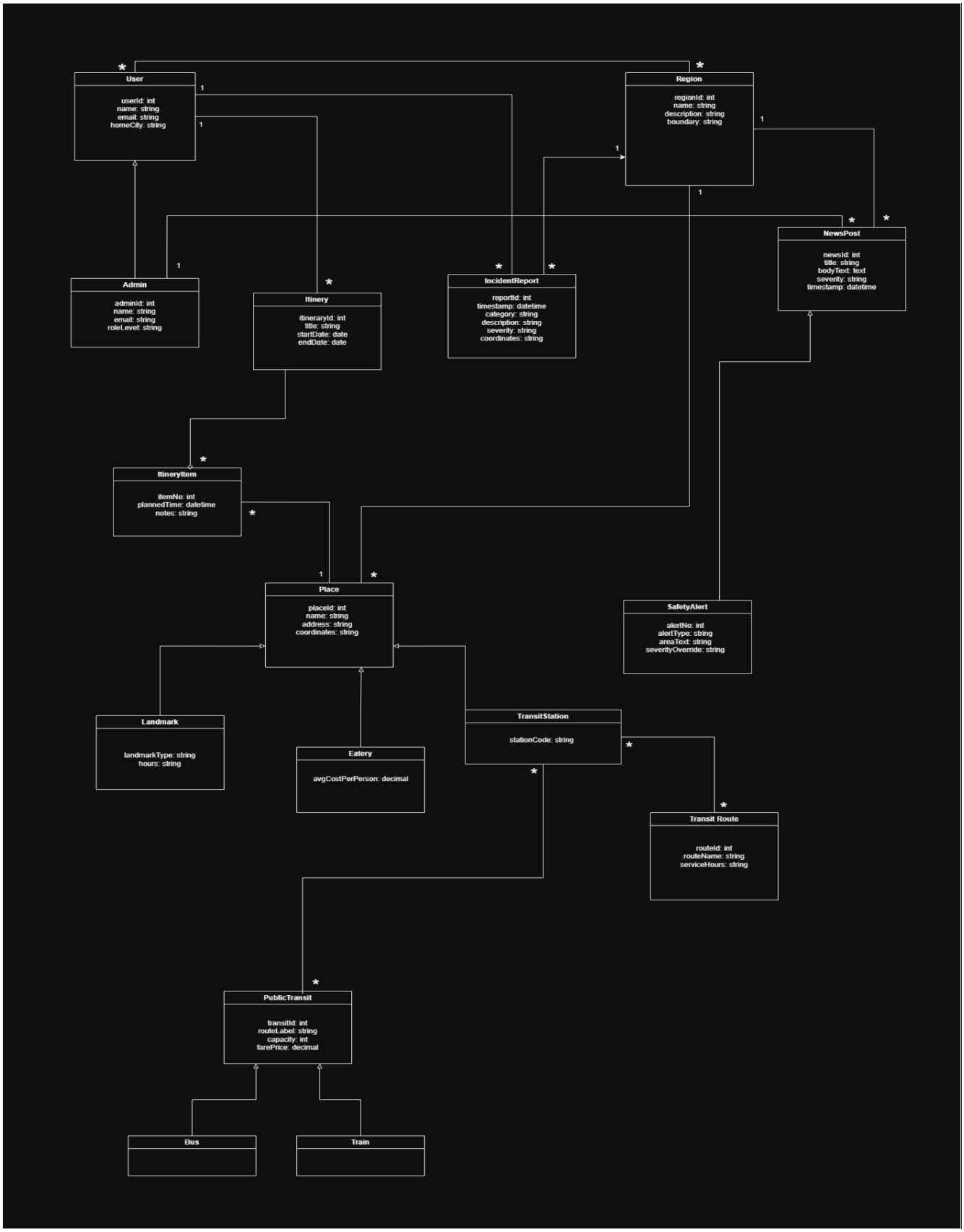

Figure 1: UML Class Diagram for Urban Scout

Section 2: End-User Functionalities

User

- Subscribe to specific `Regions` to access services and data within those `Regions`
- Queries made for the `User` are scoped to their subscribed `Regions` for efficiency
- Create and manage personal `Itinerary` entries
    - Add `ItineraryItem` entries that reference specific `Places` within subscribed `Regions`
    - Browse the `Place` table to discover `Landmark`, `Eatery`, and `TransitStation` entries
- Submit `IncidentReport` entries for subscribed `Regions`
    - Reports are forwarded to `Admin` users for verification and approval
- View official `NewsPost` entries published by `Admin` users for their subscribed `Regions`

Admin

- Verify and approve `IncidentReport` entries submitted by `User` accounts
    - Update verification status via the `Verifies` table (pending, confirmed, rejected)
- Publish official `NewsPost` entries to specific `Regions`
    - `NewsPost` entries may be created based on verified `User` `IncidentReport` submissions
    - Each `NewsPost` is associated with a `Region` and may include `SafetyAlert` entries

System (Web Application)

- Generate transit route plans for users based on `TransitRoute`, `PublicTransit`, and `TransitStation` data
- Provide proximity-based recommendations linking `TransitStation` entries to nearby `Place` entries (`Landmark`, `Eatery`)

User-Admin Interaction Flow:

A `User` submits an `IncidentReport` for a subscribed `Region`. An `Admin` assigned to that `Region` reviews and verifies the report via the `Verifies` table. Based on confirmed reports, the `Admin` publishes a `NewsPost` (optionally with a `SafetyAlert`) visible to all `User`s subscribed to that `Region`.

Section 3: SQL and DDL Statements

```
-- Urban Scout - Unified Mobile Tourist Assistant
-- CPSC 471 - Group 61 - Progress Report 3
-- SQL DDL & DML Statements
```

CREATE DATABASE & CREATE TABLE

```
-- CREATE DATABASE
CREATE DATABASE IF NOT EXISTS UrbanScout;
USE UrbanScout;

-- CREATE TABLE statements

-- ----- Core User Tables -----

CREATE TABLE User (
    UserID      INT             PRIMARY KEY AUTO_INCREMENT,
    name        VARCHAR(100)    NOT NULL,
    email       VARCHAR(150)    NOT NULL UNIQUE,
    home_city   VARCHAR(100)
);

CREATE TABLE Region (
    RegionID    INT             PRIMARY KEY AUTO_INCREMENT,
    name        VARCHAR(100)    NOT NULL,
    description TEXT,
    boundary    VARCHAR(255)
);

CREATE TABLE Admin (
    AdminID     INT             PRIMARY KEY AUTO_INCREMENT,
    name        VARCHAR(100)    NOT NULL,
    email       VARCHAR(150)    NOT NULL UNIQUE,
    role_level  VARCHAR(50),
    RegionID    INT             NOT NULL,
    FOREIGN KEY (RegionID) REFERENCES Region(RegionID)
        ON DELETE RESTRICT ON UPDATE CASCADE
);

-- ----- Place & Specializations (disjoint, partial) -----

CREATE TABLE Place (
    PlaceID     INT             PRIMARY KEY AUTO_INCREMENT,
    name        VARCHAR(150)    NOT NULL,
    address     VARCHAR(255),
    coordinates VARCHAR(50)
);

CREATE TABLE Landmark (
    LandmarkID      INT         PRIMARY KEY,
    landmark_type   VARCHAR(50),
    hours           VARCHAR(100),
    FOREIGN KEY (LandmarkID) REFERENCES Place(PlaceID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE Eatery (
    EateryID            INT             PRIMARY KEY,
    avg_cost_per_person DECIMAL(8,2),
    FOREIGN KEY (EateryID) REFERENCES Place(PlaceID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE TransitStation (
    StationID       INT             PRIMARY KEY,
    station_code    VARCHAR(20),
```

```
    FOREIGN KEY (StationID) REFERENCES Place(PlaceID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- ----- Transit Tables -----

CREATE TABLE TransitRoute (
    RouteID         INT             PRIMARY KEY AUTO_INCREMENT,
    route_name      VARCHAR(100)    NOT NULL,
    service_hours   VARCHAR(100)
);

CREATE TABLE PublicTransit (
    TransitID       INT             PRIMARY KEY AUTO_INCREMENT,
    route_label     VARCHAR(50),
    capacity        INT,
    fare_price      DECIMAL(6,2),
    vehicle_type    BOOLEAN
);

-- ----- News & Safety -----

CREATE TABLE NewsPost (
    NewsID      INT             PRIMARY KEY AUTO_INCREMENT,
    title       VARCHAR(200)    NOT NULL,
    severity    VARCHAR(20),
    time_stamp  DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
    body_text   TEXT,
    RegionID    INT             NOT NULL,
    AdminID     INT             NOT NULL,
    FOREIGN KEY (RegionID) REFERENCES Region(RegionID)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (AdminID)  REFERENCES Admin(AdminID)
        ON DELETE RESTRICT ON UPDATE CASCADE
);

-- SafetyAlert is a WEAK ENTITY identified by (NewsID, AlertNo)
CREATE TABLE SafetyAlert (
    NewsID              INT         NOT NULL,
    AlertNo             INT         NOT NULL,
    alert_type          VARCHAR(50),
    area_text           VARCHAR(255),
    severity_override   VARCHAR(20),
    PRIMARY KEY (NewsID, AlertNo),
    FOREIGN KEY (NewsID) REFERENCES NewsPost(NewsID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- ----- Incident Reports -----

CREATE TABLE IncidentReport (
    Report_ID   INT             PRIMARY KEY AUTO_INCREMENT,
    timestamp   DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
    category    VARCHAR(50),
    description TEXT,
    severity    VARCHAR(20),
    coordinates VARCHAR(50),
    UserID      INT             NOT NULL,
    RegionID    INT             NOT NULL,
    FOREIGN KEY (UserID)   REFERENCES User(UserID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (RegionID) REFERENCES Region(RegionID)
        ON DELETE RESTRICT ON UPDATE CASCADE
);

-- ----- Itinerary & Items -----
```

```
CREATE TABLE Itinerary (
    ItineraryID INT             PRIMARY KEY AUTO_INCREMENT,
    title       VARCHAR(150)    NOT NULL,
    start_date  DATE,
    end_date    DATE,
    UserID      INT             NOT NULL,
    FOREIGN KEY (UserID) REFERENCES User(UserID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- ItineraryItem is a weak entity of Itinerary
CREATE TABLE ItineraryItem (
    ItineraryID INT         NOT NULL,
    ItemNo      INT         NOT NULL,
    planned_time DATETIME,
    notes       TEXT,
    PRIMARY KEY (ItineraryID, ItemNo),
    FOREIGN KEY (ItineraryID) REFERENCES Itinerary(ItineraryID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- ----- Junction / Relationship Tables -----

-- References: links ItineraryItem to a Place
CREATE TABLE `References` (
    PlaceID     INT     NOT NULL,
    ItineraryID INT     NOT NULL,
    ItemNo      INT     NOT NULL,
    PRIMARY KEY (PlaceID, ItineraryID, ItemNo),
    FOREIGN KEY (PlaceID)     REFERENCES Place(PlaceID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (ItineraryID, ItemNo) REFERENCES ItineraryItem(ItineraryID, ItemNo)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Subscribes_to: User subscribes to Regions (M:N)
CREATE TABLE Subscribes_to (
    UserID      INT     NOT NULL,
    RegionID    INT     NOT NULL,
    PRIMARY KEY (UserID, RegionID),
    FOREIGN KEY (UserID)   REFERENCES User(UserID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (RegionID) REFERENCES Region(RegionID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Views: User views NewsPost (M:N)
CREATE TABLE Views (
    UserID      INT     NOT NULL,
    NewsID      INT     NOT NULL,
    PRIMARY KEY (UserID, NewsID),
    FOREIGN KEY (UserID) REFERENCES User(UserID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (NewsID) REFERENCES NewsPost(NewsID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Verifies: Admin verifies IncidentReport
CREATE TABLE Verifies (
    AdminID     INT         NOT NULL,
    ReportID    INT         NOT NULL,
    Status      VARCHAR(30) NOT NULL DEFAULT 'pending',
    PRIMARY KEY (AdminID, ReportID),
    FOREIGN KEY (AdminID)  REFERENCES Admin(AdminID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (ReportID) REFERENCES IncidentReport(Report_ID)
        ON DELETE CASCADE ON UPDATE CASCADE
);
```

```sql
-- Services: proximity between Place and TransitStation
CREATE TABLE Services (
    PlaceID     INT             NOT NULL,
    StationID   INT             NOT NULL,
    distance_m  DECIMAL(8,1),
    PRIMARY KEY (PlaceID, StationID),
    FOREIGN KEY (PlaceID)   REFERENCES Place(PlaceID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (StationID) REFERENCES TransitStation(StationID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Serves: TransitRoute served by PublicTransit at a TransitStation
CREATE TABLE Serves (
    RouteID     INT     NOT NULL,
    StationID   INT     NOT NULL,
    TransitID   INT     NOT NULL,
    PRIMARY KEY (RouteID, StationID, TransitID),
    FOREIGN KEY (RouteID)   REFERENCES TransitRoute(RouteID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (StationID) REFERENCES TransitStation(StationID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (TransitID) REFERENCES PublicTransit(TransitID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Stops_At: PublicTransit stops at a TransitStation
CREATE TABLE Stops_At (
    StationID   INT     NOT NULL,
    TransitID   INT     NOT NULL,
    PRIMARY KEY (StationID, TransitID),
    FOREIGN KEY (StationID) REFERENCES TransitStation(StationID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (TransitID) REFERENCES PublicTransit(TransitID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Arrival_times: scheduled arrivals for PublicTransit at a TransitStation
CREATE TABLE Arrival_times (
    TransitID       INT     NOT NULL,
    StationID       INT     NOT NULL,
    arrival_time    TIME    NOT NULL,
    PRIMARY KEY (TransitID, StationID, arrival_time),
    FOREIGN KEY (TransitID) REFERENCES PublicTransit(TransitID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (StationID) REFERENCES TransitStation(StationID)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Departure_times: scheduled departures for PublicTransit at a TransitStation
CREATE TABLE Departure_times (
    TransitID       INT     NOT NULL,
    StationID       INT     NOT NULL,
    departure_time  TIME    NOT NULL,
    PRIMARY KEY (TransitID, StationID, departure_time),
    FOREIGN KEY (TransitID) REFERENCES PublicTransit(TransitID)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (StationID) REFERENCES TransitStation(StationID)
        ON DELETE CASCADE ON UPDATE CASCADE
);
```

INSERT -- Sample Data

```sql
-- INSERT - Sample Data (maintains referential integrity)
```

```sql
-- Regions
INSERT INTO Region (RegionID, name, description, boundary) VALUES
(1, 'Downtown Seoul',     'Central business and tourist district',     'Jongno-gu / Jung-gu'),
(2, 'Gangnam',            'Southern Seoul commercial district',        'Gangnam-gu'),
(3, 'Hongdae',            'University area known for nightlife & art', 'Mapo-gu');

-- Users
INSERT INTO User (UserID, name, email, home_city) VALUES
(1, 'Alice Park',    'alice@example.com',    'Calgary'),
(2, 'Bob Kim',       'bob@example.com',      'Vancouver'),
(3, 'Charlie Lee',   'charlie@example.com',  'Toronto');

-- Admins (each assigned to one region)
INSERT INTO Admin (AdminID, name, email, role_level, RegionID) VALUES
(1, 'Soyeon Choi',   'soyeon@urbanscout.io',  'senior',   1),
(2, 'Minho Kang',    'minho@urbanscout.io',   'junior',   2),
(3, 'Jisoo Yoon',    'jisoo@urbanscout.io',   'senior',   3);

-- Places
INSERT INTO Place (PlaceID, name, address, coordinates) VALUES
(1, 'Gyeongbokgung Palace',   '161 Sajik-ro, Jongno-gu',        '37.5796,126.9770'),
(2, 'Namsan Tower',            '105 Namsangongwon-gil, Yongsan',  '37.5512,126.9882'),
(3, 'Myeongdong Kyoja',        '29 Myeongdong 10-gil, Jung-gu',   '37.5636,126.9856'),
(4, 'Maple Tree House',        '7 Apgujeong-ro 46-gil, Gangnam',  '37.5273,127.0366'),
(5, 'Seoul Station',           '405 Hangang-daero, Yongsan-gu',   '37.5547,126.9707'),
(6, 'Hongik Univ. Station',    '188 Yanghwa-ro, Mapo-gu',         '37.5571,126.9246'),
(7, 'Gangnam Station',         '396 Gangnam-daero, Gangnam-gu',   '37.4979,127.0276');

-- Landmarks
INSERT INTO Landmark (LandmarkID, landmark_type, hours) VALUES
(1, 'Palace',       '09:00-18:00'),
(2, 'Observation',  '10:00-23:00');

-- Eateries
INSERT INTO Eatery (EateryID, avg_cost_per_person) VALUES
(3, 9.00),
(4, 25.50);

-- Transit Stations
INSERT INTO TransitStation (StationID, station_code) VALUES
(5, 'SEL'),
(6, 'HUV'),
(7, 'GNM');

-- Transit Routes
INSERT INTO TransitRoute (RouteID, route_name, service_hours) VALUES
(1, 'Line 1 - Gyeongui-Jungang', '05:30-00:00'),
(2, 'Line 2 - Circle',           '05:30-00:00'),
(3, 'Line 4 - Blue',             '05:30-00:00');

-- Public Transit vehicles / services
INSERT INTO PublicTransit (TransitID, route_label, capacity, fare_price, vehicle_type) VALUES
(1, '1호선',  1500, 1.35, 'Subway'),
(2, '2호선',  1600, 1.35, 'Subway'),
(3, '4호선',  1400, 1.35, 'Subway'),
(4, '370',    50,   1.35, 'Bus'),
(5, 'KTX 101',800, 45.00, 'Train');

-- Bus specialization
INSERT INTO Bus (TransitID, bus_number) VALUES
(4, '370');

-- Train specialization
INSERT INTO Train (TransitID, car_count) VALUES
(5, 10);

-- News Posts
```

```
INSERT INTO NewsPost (NewsID, title, severity, time_stamp, body_text, RegionID, AdminID) VALUES
(1, 'Jongno Cherry Blossom Festival This Weekend',    'low',    '2026-03-18 10:00:00', 'Annual cherry
blossom festival along Jongno streets. Expect large crowds.', 1, 1),
(2, 'Gangnam Road Closure for Marathon',               'medium', '2026-03-19 08:00:00',
'Gangnam-daero will be closed from 6 AM to 2 PM on Saturday.',               2, 2),
(3, 'Hongdae Street Performance Ban Lifted',           'low',    '2026-03-17 14:30:00', 'Street
performances are once again permitted in the Hongdae area.',          3, 3);

-- Safety Alerts (weak entity - composite PK)
INSERT INTO SafetyAlert (NewsID, AlertNo, alert_type, area_text, severity_override) VALUES
(2, 1, 'road_closure', 'Gangnam-daero between Sinnonhyeon and Yeoksam', 'high'),
(1, 1, 'crowd',        'Gwanghwamun Square',                           NULL);

-- Incident Reports
INSERT INTO IncidentReport (Report_ID, timestamp, category, description, severity, coordinates,
UserID, RegionID) VALUES
(1, '2026-03-18 15:22:00', 'theft',      'Wallet stolen near Myeongdong shopping street.',
'medium', '37.5636,126.9860', 1, 1),
(2, '2026-03-19 09:05:00', 'road_hazard','Large pothole on sidewalk near Gangnam Station exit
3.','low',    '37.4981,127.0278', 2, 2);

-- Itineraries
INSERT INTO Itinerary (ItineraryID, title, start_date, end_date, UserID) VALUES
(1, 'Seoul Weekend Trip',      '2026-03-21', '2026-03-23', 1),
(2, 'Gangnam Food Tour',       '2026-03-22', '2026-03-22', 2);

-- Itinerary Items (weak entity)
INSERT INTO ItineraryItem (ItineraryID, ItemNo, planned_time, notes) VALUES
(1, 1, '2026-03-21 09:00:00', 'Visit the palace in the morning'),
(1, 2, '2026-03-21 12:30:00', 'Lunch at Myeongdong Kyoja'),
(1, 3, '2026-03-21 15:00:00', 'Namsan Tower sunset'),
(2, 1, '2026-03-22 11:00:00', 'Korean BBQ at Maple Tree House');

-- References (ItineraryItem -> Place)
INSERT INTO `References` (PlaceID, ItineraryID, ItemNo) VALUES
(1, 1, 1),
(3, 1, 2),
(2, 1, 3),
(4, 2, 1);

-- Subscriptions (User subscribes to Region)
INSERT INTO Subscribes_to (UserID, RegionID) VALUES
(1, 1),
(1, 3),
(2, 2),
(3, 1),
(3, 2),
(3, 3);

-- Views (User views NewsPost)
INSERT INTO Views (UserID, NewsID) VALUES
(1, 1),
(1, 2),
(2, 2),
(3, 3);

-- Verifies (Admin verifies IncidentReport)
INSERT INTO Verifies (AdminID, ReportID, Status) VALUES
(1, 1, 'confirmed'),
(2, 2, 'pending');

-- Services (Place <-> TransitStation proximity)
INSERT INTO Services (PlaceID, StationID, distance_m) VALUES
(1, 5, 1200.0),
(2, 5, 2500.0),
(3, 5,  800.0),
(4, 7,  450.0),
```

```
(1, 6, 4500.0);

-- Serves (TransitRoute <-> TransitStation <-> PublicTransit)
INSERT INTO Serves (RouteID, StationID, TransitID) VALUES
(1, 5, 1),
(2, 6, 2),
(2, 7, 2),
(3, 5, 3);

-- Stops_At
INSERT INTO Stops_At (StationID, TransitID) VALUES
(5, 1),
(5, 3),
(6, 2),
(7, 2),
(6, 4),
(5, 5);

-- Arrival times
INSERT INTO Arrival_times (TransitID, StationID, arrival_time) VALUES
(1, 5, '06:00:00'),
(1, 5, '06:10:00'),
(2, 6, '06:05:00'),
(2, 7, '06:20:00'),
(3, 5, '06:03:00'),
(4, 6, '06:15:00'),
(5, 5, '07:00:00');

-- Departure times
INSERT INTO Departure_times (TransitID, StationID, departure_time) VALUES
(1, 5, '06:01:00'),
(1, 5, '06:11:00'),
(2, 6, '06:06:00'),
(2, 7, '06:21:00'),
(3, 5, '06:04:00'),
(4, 6, '06:16:00'),
(5, 5, '07:01:00');
```

SELECT -- Example Queries

```
-- SELECT - Example Queries

-- 1. Get all landmarks with their place info
SELECT p.PlaceID, p.name, p.address, p.coordinates,
       l.landmark_type, l.hours
FROM Place p
JOIN Landmark l ON p.PlaceID = l.LandmarkID;

-- 2. Get all eateries with cost info
SELECT p.PlaceID, p.name, p.address, e.avg_cost_per_person
FROM Place p
JOIN Eatery e ON p.PlaceID = e.EateryID;

-- 3. Find nearby transit stations for a given place (sorted by distance)
SELECT p.name AS place_name, ts_place.name AS station_name,
       ts.station_code, s.distance_m
FROM Services s
JOIN Place p        ON s.PlaceID   = p.PlaceID
JOIN Place ts_place ON s.StationID = ts_place.PlaceID
JOIN TransitStation ts ON s.StationID = ts.StationID
WHERE p.PlaceID = 1
ORDER BY s.distance_m ASC;

-- 4. Get all news posts for a region, with any safety alerts
SELECT np.NewsID, np.title, np.severity, np.time_stamp, np.body_text,
       sa.AlertNo, sa.alert_type, sa.area_text, sa.severity_override
```

```
FROM NewsPost np
LEFT JOIN SafetyAlert sa ON np.NewsID = sa.NewsID
WHERE np.RegionID = 1;

-- 5. Show a user's itinerary with places they plan to visit
SELECT i.title AS itinerary_title, ii.ItemNo, ii.planned_time, ii.notes,
       p.name AS place_name, p.address
FROM Itinerary i
JOIN ItineraryItem ii ON i.ItineraryID = ii.ItineraryID
JOIN `References` r   ON ii.ItineraryID = r.ItineraryID AND ii.ItemNo = r.ItemNo
JOIN Place p          ON r.PlaceID = p.PlaceID
WHERE i.UserID = 1
ORDER BY ii.planned_time;

-- 6. List all regions a user is subscribed to
SELECT u.name AS user_name, r.name AS region_name
FROM Subscribes_to st
JOIN User u   ON st.UserID   = u.UserID
JOIN Region r ON st.RegionID = r.RegionID
WHERE st.UserID = 1;

-- 7. Get all incident reports for a region with verification status
SELECT ir.Report_ID, ir.category, ir.severity, ir.description,
       u.name AS reported_by,
       v.Status AS verification_status, a.name AS verified_by
FROM IncidentReport ir
JOIN User u ON ir.UserID = u.UserID
LEFT JOIN Verifies v ON ir.Report_ID = v.ReportID
LEFT JOIN Admin a    ON v.AdminID    = a.AdminID
WHERE ir.RegionID = 1;

-- 8. Find all transit arrival and departure times at a specific station
SELECT pt.route_label, pt.vehicle_type,
       at.arrival_time,
       ts_place.name AS station_name
FROM PublicTransit pt
JOIN Arrival_times at ON pt.TransitID = at.TransitID
JOIN Place ts_place   ON at.StationID = ts_place.PlaceID
WHERE at.StationID = 5
ORDER BY at.arrival_time;

-- 9. Get all news posts viewed by a specific user
SELECT u.name, np.title, np.time_stamp
FROM Views v
JOIN User u      ON v.UserID = u.UserID
JOIN NewsPost np ON v.NewsID = np.NewsID
WHERE v.UserID = 1;

-- 10. Count incident reports per region
SELECT r.name AS region_name, COUNT(ir.Report_ID) AS total_reports
FROM Region r
LEFT JOIN IncidentReport ir ON r.RegionID = ir.RegionID
GROUP BY r.RegionID, r.name;
```

UPDATE -- Example Statements

```
-- UPDATE - Example Statements

-- 1. Update a user's home city
UPDATE User
SET home_city = 'Edmonton'
WHERE UserID = 1;

-- 2. Mark an incident report as verified by an admin
UPDATE Verifies
SET Status = 'confirmed'
```

```
WHERE AdminID = 2 AND ReportID = 2;

-- 3. Update the severity of a news post
UPDATE NewsPost
SET severity = 'high'
WHERE NewsID = 2;

-- 4. Change the operating hours of a landmark
UPDATE Landmark
SET hours = '08:00-20:00'
WHERE LandmarkID = 1;

-- 5. Update the average cost of an eatery
UPDATE Eatery
SET avg_cost_per_person = 10.50
WHERE EateryID = 3;
```

DELETE -- Example Statements

```
-- DELETE - Example Statements

-- 1. Remove a subscription
DELETE FROM Subscribes_to
WHERE UserID = 1 AND RegionID = 3;

-- 2. Delete a specific safety alert
DELETE FROM SafetyAlert
WHERE NewsID = 1 AND AlertNo = 1;

-- 3. Remove an itinerary item
DELETE FROM ItineraryItem
WHERE ItineraryID = 1 AND ItemNo = 3;

-- 4. Delete an incident report (cascades to Verifies)
DELETE FROM IncidentReport
WHERE Report_ID = 2;

-- 5. Remove a news post (cascades to SafetyAlert and Views)
DELETE FROM NewsPost
WHERE NewsID = 3;
```